Análisis y Diseño de Algoritmos I

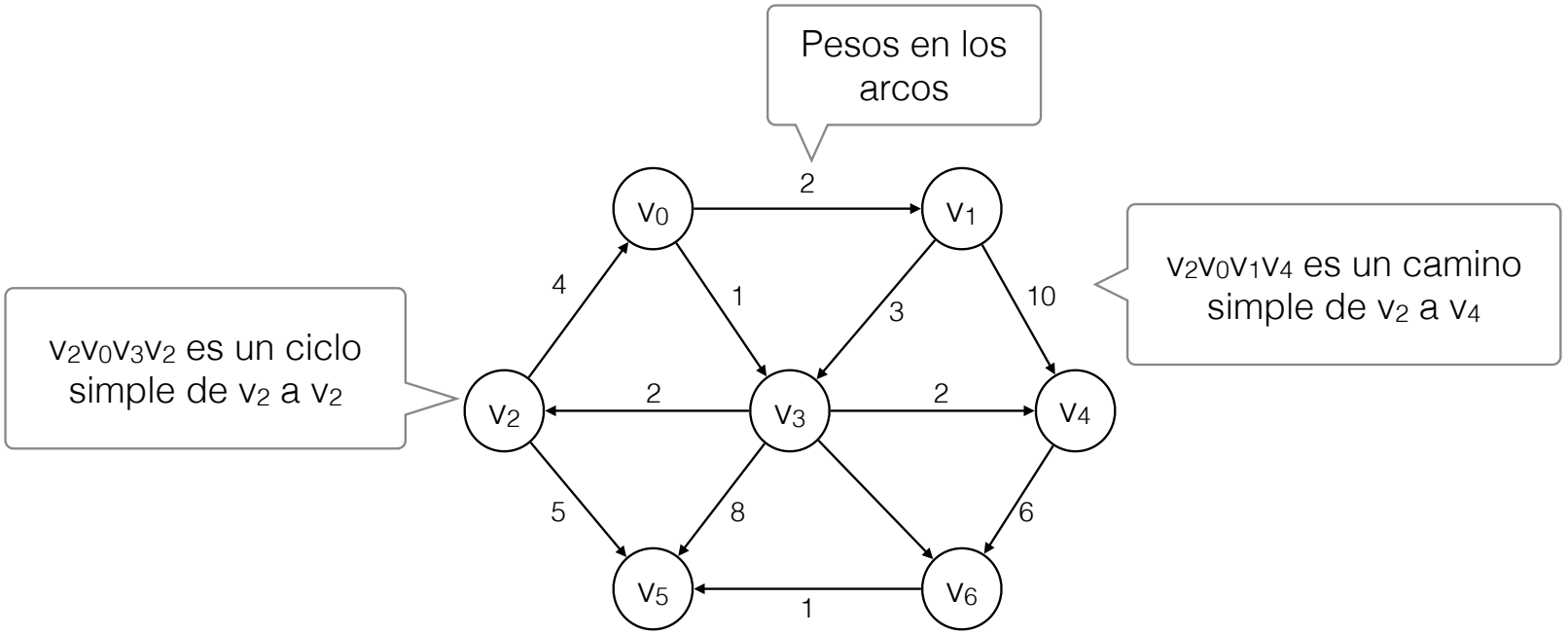
Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Algoritmos sobre Grafos y Programación Dinámica

Definiciones…

- Un grafo se dice **completo** si cada par de nodos está conectado por un arco,
- Un **camino** de un nodo v a un nodo w en un grafo G es una secuencia de nodos adyacentes que comienza en v y termina w.
- Si todos los vértices en el camino son distintos el camino se denomina **camino simple**.
- Un **ciclo** es un camino de un nodo w en si mismo.
- Un **ciclo simple** es un ciclo en el cual no se repiten arcos ni vértices (salvo el primero y el último).
- Un grafo se llama **conexo** si existe un camino desde v a w, para cada par de nodos v y w.
- Un grafo con **pesos** es un grafo en el cual los arcos poseen pesos y costos.

Un Ejemplo



Este es un grafo dirigido (**digrafo**)

Representación de Grafos

Dos formas básicas de representar grafos:

- **Matrices**: Para un grafo de N vertices se crea una matriz G de NxN, en donde G[i][j]=1 si el nodo i está conectado con el nodo j, y G[i][j]=0 en otro caso.
- **Lista de adyacencias:** Se crea un arreglo G de N lugares, en donde G[i] con tiene una lista con todos los adyacentes de i.

$O(N^2)$ espacio

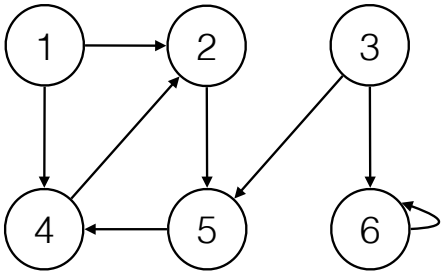
Saber si dos nodos están conectados es O(1)

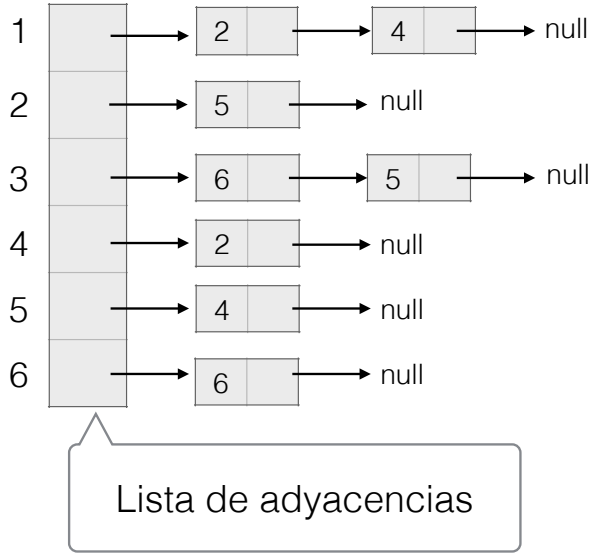
O(|V|+|E|) espacio

Saber si dos nodos están conectados es O(N)

Ejemplo

Supongamos el siguiente grafo:





G

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Matriz de adyacencia

Recorridos de Grafos

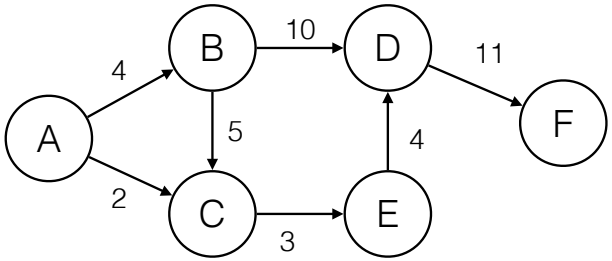
Dos formas básicas de recorrer grafos, que permiten implementar muchos algoritmos sobre estos:

- **Depth-First Search**: o primero en profundidad, se recorre el grafo tal que siempre nos metemos por un camino antes de recorrer los otros.

- **Breadth-First Search**: o primero en amplitud, se recorren todos los adyacentes a un nodo primero, y luego se elige un nodo y se continua de esta forma

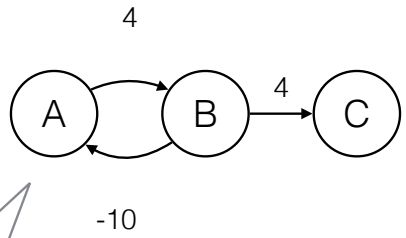
Caminos más Cortos

Uno de los problemas más frecuentes en grafos es calcular el camino más corto entre dos nodos.



Grafo dirigido con costos en los arcos

Camino más corto entre A y F tiene costo 20



Cuando hay costos negativos el camino más corto puede no existir

Algoritmo de Warshall-Floyd

El algoritmo de Warshall permite calcular los caminos más cortos entre cada par de nodos de un grafo.

Idea Recursiva:

Consideramos que los nodos están enumerados

warshall i j k calcula el camino más corto de i a j pasando por los primeros k nodos

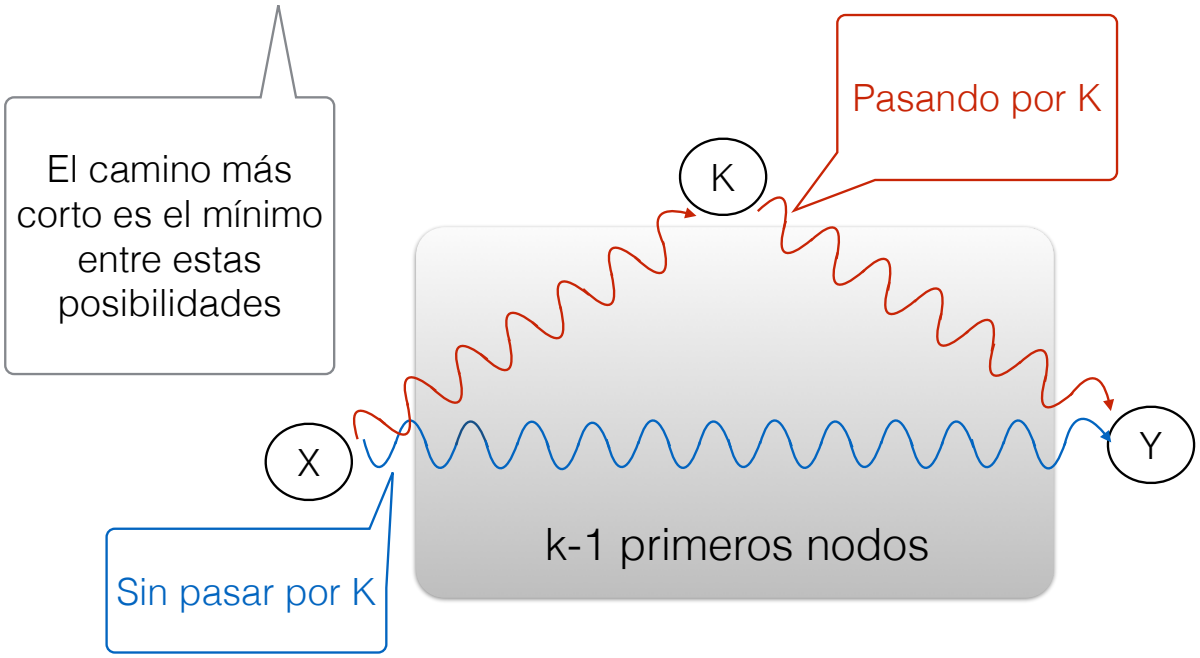
$$warshall : Nat \to Nat \to Nat \to Nat$$
$$warshall\ i\ j\ 0 = w(i,j)$$
$$warshall\ i\ j\ k = min(warshall(i,j,k-1), warshall(i,k,k-1) + warshall(k,j,k-1))$$

O(3^n)

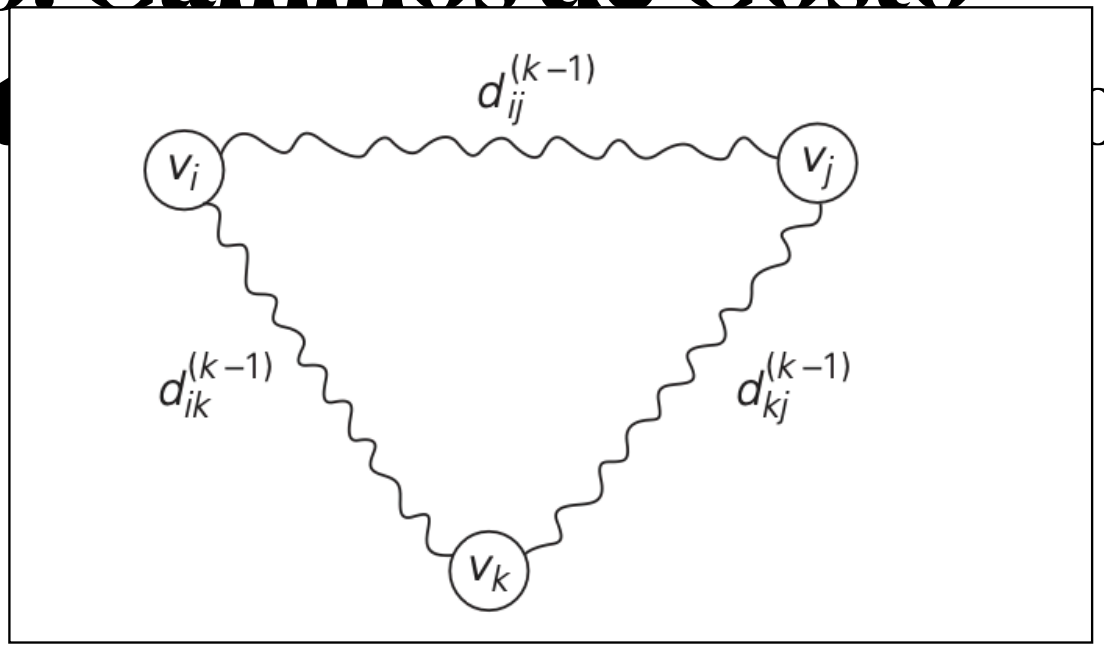
el mínimo entre ir de i a j por los primeros k-1, o ir de i a k por los primeros k-1 y después de k a j por los k-1 primeros

Idea

El camino más corto de X a Y pasando por los k primeros nodos, puede ser, o bien pasar por k o bien sin pasar por k (y pasando por los nodos [0..k-1])



Ejemplo: Caminos de Costo Mínimo ent



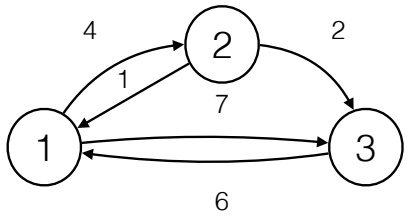
$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} \quad \text{for } k \geq 1,\ \ d_{ij}^{(0)} = w_{ij}.$$

Caminos de Costo Mínimo entre Vértices de un Grafo con PD

El algoritmo anterior puede resolverse usando Programación Dinámica de manera mucho más eficiente. El pseudo-código de un algoritmo basado en Programación Dinámica y la solución recursiva anterior es el siguiente:

```
ALGORITHM Floyd(W[1..n, 1..n])
    //Implements Floyd's algorithm for the all-pairs shortest-paths problem
    //Input: The weight matrix W of a graph with no negative-length cycle
    //Output: The distance matrix of the shortest paths' lengths
    D ← W //is not necessary if W can be overwritten
    for k ← 1 to n do
        for i ← 1 to n do
            for j ← 1 to n do
                D[i, j] ← min{D[i, j], D[i, k] + D[k, j]}
    return D
```

Ejemplo




```
ALGORITHM Floyd(W[1..n, 1..n])

    D ← W //is not necessary if W can be overwritten
    for k ← 1 to n do
        for i ← 1 to n do
            for j ← 1 to n do
                D[i, j] ← min{D[i, j], D[i, k] + D[k, j]}
    return D
```

Dist⁰=

	1	2	3
1	0	4	7
2	1	0	2
3	6	inf	0

inic.

Dist¹=

	1	2	3
1	0	4	7
2	1	0	2
3	6	10	0

k=1

Fila 1 (i = 1)

- D[1,1] = min (D[1,1], D[1,1] + D[1,1])
- D[1,2] = min (D[1,2], D[1,1] + D[1,2])
- D[1,3] = min (D[1,3], D[1,1] + D[1,3])

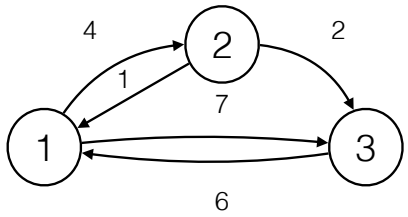
Fila 2 (i = 2)

- D[2,1] = min (D[2,1], D[2,1] + D[1,1])
- D[2,2] = min (D[2,2], D[2,1] + D[1,2])
- D[2,3] = min (D[2,3], D[2,1] + D[1,3])

Fila 3 (i = 3)

- D[3,1] = min (D[3,1], D[3,1] + D[1,1])
- **D[3,2] = min (D[3,2], D[3,1] + D[1,2])**
- D[3,3] = min (D[3,3], D[3,1] + D[1,3])

Ejemplo



ALGORITHM *Floyd*$(W[1..n, 1..n])$

```
//Implements Floyd's algorithm for the all-pairs shortest-paths problem
//Input: The weight matrix W of a graph with no negative-length cycle
//Output: The distance matrix of the shortest paths' lengths
D ← W //is not necessary if W can be overwritten
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            D[i, j] ← min{D[i, j], D[i, k] + D[k, j]}
return D
```

$Dist^0$=

	1	2	3
1	0	4	7
2	1	0	2
3	6	inf	0

inic.

$Dist^1$=

	1	2	3
1	0	4	7
2	1	0	2
3	6	10	0

k=1

$Dist^2$=

	1	2	3
1	0	4	6
2	1	0	2
3	6	10	0

k=2

$Dist^3$=

	1	2	3
1	0	4	6
2	1	0	2
3	6	10	0

k=3

$Path^0$=

	1	2	3
1	-1	1	1
2	2	-1	2
3	3	-1	-1

inic.

$Path^1$=

	1	2	3
1	-1	1	1
2	2	-1	2
3	3	1	-1

k=1

$Path^2$=

	1	2	3
1	-1	1	2
2	2	-1	2
3	3	1	-1

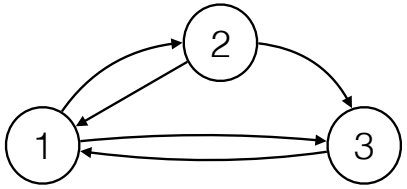
k=2

$Path^3$=

	1	2	3
1	-1	1	2
2	2	-1	2
3	3	1	-1

Grafos como Matrices

Cuando no hay costos podemos ver a los grafos como matrices Booleanas (relaciones)



Por ejemplo:

0	1	1
1	0	1
1	0	0

La clausura reflexo-transitiva de un grafo G es la minima relación reflexiva y transitiva que incluye a G, se nota: G^*

$(x, y) \in G^*$ ssi hay un camino de x a y en el grafo

Calculando la Clausura Transitiva

Podemos calcular la clausura transitiva con algebra matricial:

$$G^* = I + G + G^2 + G^3 \ldots$$

También podemos modificar Warshall para calcular la clausura transitiva:

```
for(int k = 1 ; k < n; k++){
        for (int i=1; i<n; i++){
            for (int j=1; j<n; j++){
               d[i][j] = d[i][j] || (d[i][k] && d[k][j])
             }
        }
 }
```